O caminho dos dados

GET, POST, HTTPS e contrato do projeto integrador

Curso Técnico em Informática

Prof. Fabrício Rodrigues

O que vamos construir

Distinguir uma simulação local de uma requisição a um serviço.

Comparar GET e POST e explicar o papel do HTTPS.

Documentar campos e regras de um serviço futuro sem inventar um backend.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Abra o projeto deste encontro no VS Code. Salve uma cópia própria antes de alterar.

Mapa da aula

Situação	Representação
Busca de dados com GET	Parâmetros podem aparecer na URL.
Envio com POST	Dados de formulário podem seguir no corpo.
HTTPS	Protege o tráfego em trânsito.
Projeto no GitHub Pages	Site estático; não implementa recebimento de chamados.

Os limites do protótipo

Até aqui o JavaScript trabalha somente na página. Mostrar um resumo não faz uma requisição de envio. O GitHub Pages publica os arquivos estáticos do curso; não executa um serviço próprio para receber chamados.

Uma aplicação real precisaria de uma rota de servidor e de regras de recebimento. Nesta aula documentaremos esse contrato e compararemos representações, sem enviar dados a um endereço externo.

GET e os parâmetros da URL

GET costuma ser usado para recuperar informações. Em um formulário com method=get, os pares de dados podem compor a query da URL. Eles podem aparecer no histórico e ser compartilhados junto com o endereço.

O exemplo gera uma string com URLSearchParams e a mostra como texto. exemplo.test é um domínio de exemplo; o projeto não navega para ele. Observe como nomes, espaços e caracteres são representados.

Exemplo: GET e os parâmetros da URL

```
const dados = new FormData(form);
const parametros = new URLSearchParams(dados);
const exemplo = 'https://exemplo.test/chamados?'
  + parametros.toString();
// Apenas exibir o texto. Não navegar para esse endereço.
```

Observe o papel de cada linha e compare com o arquivo do projeto.

POST, HTTPS e conferência no servidor

Em um formulário HTML com method=post, os dados podem seguir no corpo da requisição. POST sozinho não criptografa dados. HTTPS protege o tráfego em trânsito. Não coloque informações sensíveis em uma URL por achar que o método resolveu toda a proteção.

O servidor deve conferir formato, tamanho, escolhas permitidas e demais regras. O usuário pode alterar a página ou enviar outra requisição. A validação HTML e o JavaScript servem à experiência local e precisam de conferência no serviço real.

Contrato do integrador

Documente no HTML um bloco de contrato: finalidade, método escolhido, rota proposta, campos e regras a aplicar no servidor. Identifique claramente que a rota não está implementada.

A base exibe exemplos de URL GET e de um corpo para POST depois da simulação. São representações de estudo. Complete o contrato e confronte seus nomes de campos com o que FormData realmente coleta.

Exemplo: Contrato do integrador

```
<!-- Exemplo de contrato para uma aplicação futura.
Método: POST. Rota proposta: /chamados.
O servidor deve validar os campos novamente.
Este projeto estático não implementa essa rota.
-->
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Atividade no VS Code

Abrir a base da aula e observar o resumo e as representações de estudo.

Comparar onde os valores apareceriam em GET e POST e explicar por que não houve envio real.

Atividade: conferência e explicação

Completar o contrato no HTML: finalidade, POST, rota proposta /chamados, campos, regras e indicação de serviço ainda não implementado.

Conferir se os names coletados correspondem ao contrato e registrar o escopo do integrador.

Conferência dos testes

Caso	Resultado esperado
Simulação com dados fictícios	Resumo e representações locais, sem envio.
Espaços no nome	URL de exemplo com parâmetros codificados.
Dois sintomas selecionados	Representação de POST preserva a lista.
Contrato sem indicar implementação futura	Revisar para não prometer serviço existente.
POST sem HTTPS no argumento	Explicar que POST não oferece criptografia.

Conferência dos testes (continuação)

Caso	Resultado esperado
Campo sem name	Identificar a ausência na coleta e corrigir o contrato.

Erros que merecem atenção

O corpo de POST não é uma garantia de criptografia.

Não afirmar que o Pages recebeu, validou ou armazenou o chamado.

Um contrato é uma proposta; não é prova de que o endpoint existe.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

Qual parte do seu projeto roda no navegador? O que ainda faltaria num serviço real?

Guarde a versão desta aula: ela será o ponto de partida para a continuidade.